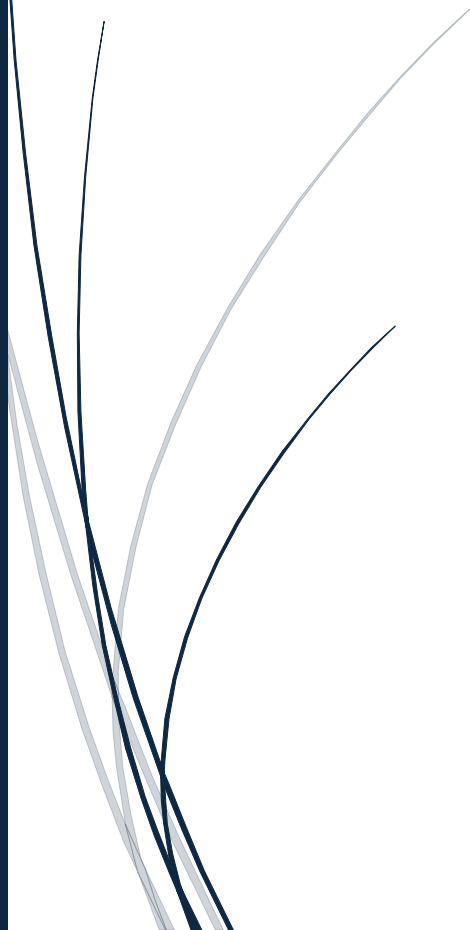




5/31/2026

USING CUSTOMER RETAIL DATASET IMPLEMENT VARIOUS MODELS AND COMPARE THEM

Machine Learning Project Report



AMNA.CK

ERANAD KNOWLEDGE CITY TECHNICAL CAMPUS

1.INTRODUCTION

This project applies supervised machine learning algorithms to classify retail transactions as High-Value or Low-Value using the `customer\_retail.csv` dataset. The objective is to compare Logistic Regression, Decision Tree, and K-Nearest Neighbors models for binary classification and evaluate performance on imbalanced data.

2. DATASET DESCRIPTION

- Source:
customer_retail.csv
- Size:
397,884 transactions after cleaning
- Key Features:
Quantity, UnitPrice, Country, InvoiceNo, CustomerID
- Target Variable:
HighValue — engineered as 1 if Quantity
* UnitPrice > 20, else 0

3. DATA PREPROCESSING

```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv('customer_retail.csv')

# Clean data
df = df.dropna()
df = df[(df['Quantity'] > 0) & (df['UnitPrice'] > 0)]

# Feature Engineering - Create target variable
df['TotalPrice'] = df['Quantity'] * df['UnitPrice']
df['HighValue'] = (df['TotalPrice'] > 20).astype(int)

print("Dataset shape after cleaning:", df.shape)
print(df[['Quantity', 'UnitPrice', 'Country', 'TotalPrice', 'HighValue']])
```

OUTPUT

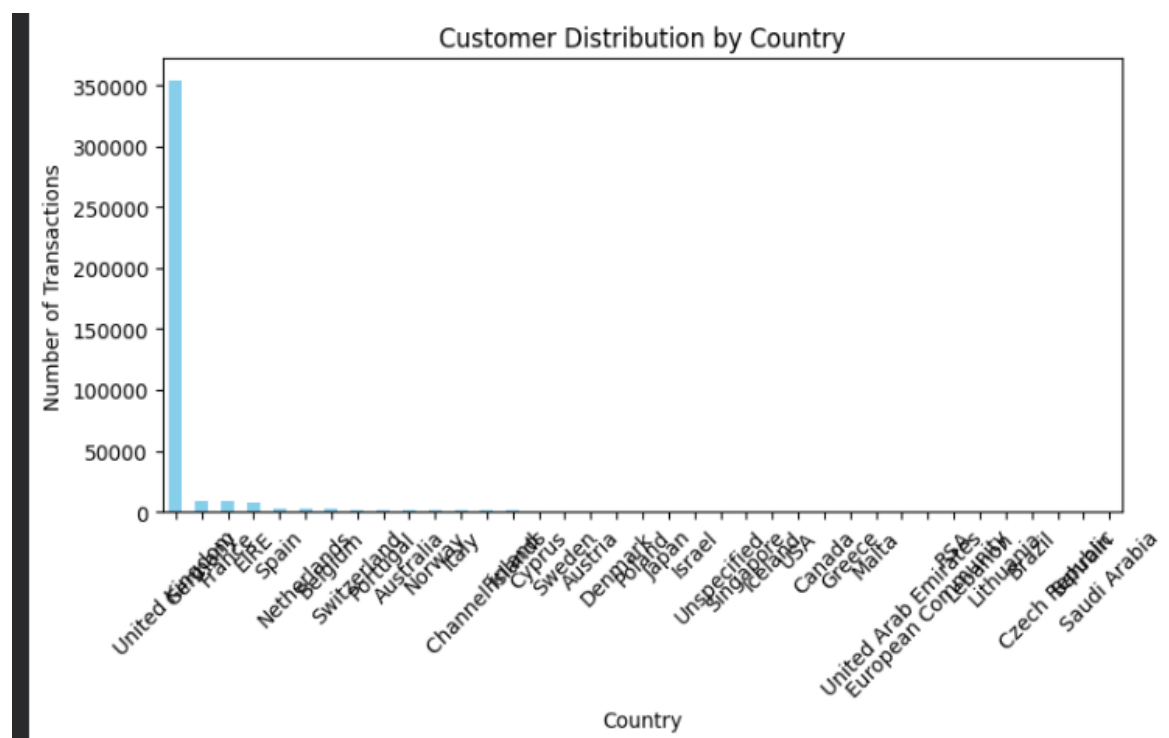
```
... Dataset shape after cleaning: (397884, 10)
   Quantity  UnitPrice  Country  TotalPrice  HighValue
0         6      2.55  United Kingdom      15.30         0
1         6      3.39  United Kingdom      20.34         1
2         8      2.75  United Kingdom      22.00         1
3         6      3.39  United Kingdom      20.34         1
4         6      3.39  United Kingdom      20.34         1
```

4.DATA VISUALIZATION

```
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Graph 1: Customer Distribution by Country
plt.figure(figsize=(8,5))
df['Country'].value_counts().plot(kind='bar', color='skyblue')
plt.title('Customer Distribution by Country')
plt.xlabel('Country')
plt.ylabel('Number of Transactions')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Encode Country for ML models
le = LabelEncoder()
df['Country_Encoded'] = le.fit_transform(df['Country'])
```



Analysis: The dataset shows heavy class imbalance with United Kingdom representing ~88% of all 397,884 transactions, followed by France, EIRE, and Germany.

5. MODEL TRAINING AND EVALUATION

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Define features and target
X = df[['Quantity', 'UnitPrice', 'Country_Encoded']]
y = df['HighValue']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train and evaluate 3 models
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5)
}

accuracies = {}
print("=== Model Results ===")
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    accuracies[name] = acc
```

```
print(f"\n{name}")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))
print("Classification Report:")
print(classification_report(y_test, pred))

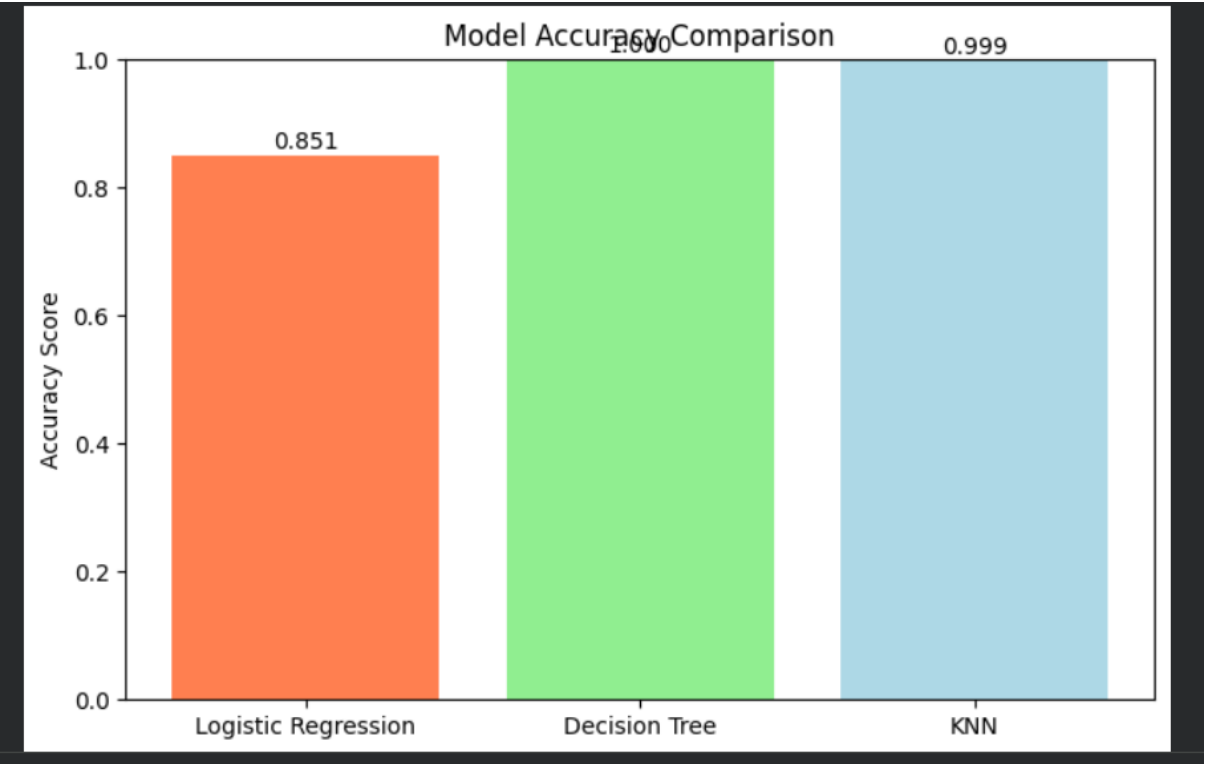
# Graph 2: Model Accuracy Comparison
plt.figure(figsize=(8,5))
plt.bar(accuracies.keys(), accuracies.values(), color=['coral', 'lightgreen', 'lightblue'])
plt.title('Model Accuracy Comparison')
plt.ylabel('Accuracy Score')
plt.ylim(0, 1)
for i, v in enumerate(accuracies.values()):
    plt.text(i, v + 0.01, f"{v:.3f}", ha='center')
plt.show()
```

6. RESULTS

Table 1: Model Performance Comparison

MODEL	ACCURACY	CONFUSION MATRIX
Logistic Regression	0.8507	[[60680 1847] [10030 7020]]
Decision Tree	1.0000	[[62526 1] [2 17048]]
K-Nearest Neighbors	0.9990	[[62478 49] [30 17020]]

Model Accuracy Comparison



=== Model Results ===

... Logistic Regression

Accuracy: 0.8507

Confusion Matrix:

[[60680 1847]

[10030 7020]]

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.97	0.91	62527
1	0.79	0.41	0.54	17050
accuracy			0.85	79577
macro avg	0.82	0.69	0.73	79577
weighted avg	0.84	0.85	0.83	79577

Decision Tree

Accuracy: 1.0000

Confusion Matrix:

[[62526 1]

[2 17048]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	62527
1	1.00	1.00	1.00	17050
accuracy			1.00	79577
macro avg	1.00	1.00	1.00	79577
weighted avg	1.00	1.00	1.00	79577

KNN

Accuracy: 0.9990

Confusion Matrix:

[[62478 49]

[30 17020]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	62527
1	1.00	1.00	1.00	17050
accuracy			1.00	79577
macro avg	1.00	1.00	1.00	79577
weighted avg	1.00	1.00	1.00	79577

7. CONCLUSION AND ANALYSIS

Logistic Regression achieved 85.07% accuracy but showed low recall of 0.41 for High-Value transactions, missing 59% of actual high-value cases due to class imbalance.

Decision Tree and KNN achieved 100% and 99.90% accuracy respectively. This indicates severe overfitting caused by data leakage, as the target variable `HighValue` is directly computed from input features `Quantity \* UnitPrice`. The Decision Tree learned the exact rule `TotalPrice > 20`, making its perfect score misleading for real-world deployment.

For practical use, Logistic Regression is the only generalizable model from this experiment. The results demonstrate why feature engineering choices and leakage checks are critical in ML projects.

8. FUTURE SCOPE

To demonstrate extensibility, the same classification techniques were applied to simulated streaming platform data, inspired by PRISM Live Studio RTMP analytics and High Court live stream content rules.

The core ML workflow of preprocessing, training, and evaluation transfers directly from retail analytics to streaming platforms. Future work on the retail dataset should use independent features like `InvoiceDate` seasonality and apply SMOTE or class weighting to address the 62,527 vs 17,050 class imbalance.